

Domain Kit Development Guide

The gert Project

April 26, 2026

Contents

1	Introduction	1
1.1	What Are Domain Kits?	1
1.2	Why Domain Kits Exist	2
1.3	Relationship to Gert Core	2
1.4	The Three Extension Points	3
1.5	Who Should Read This Guide	4
1.6	How This Guide Is Structured	5
1.7	Forward Reference	5
2	Kit Anatomy	6
2.1	Directory Structure	6
2.2	The Kit Manifest	7
2.3	Kit Versioning Contract	9
2.4	Example Kit: <code>gert.compliance</code>	11
3	Authoring Schemas	12
3.1	What Kit Schemas Are	12
3.2	Defining a Custom Step Type	12
3.3	Field Extension Conventions	14
3.4	Validation at Parse Time vs. Plan Time	15
3.5	Example: <code>evidence-capture</code> Step Type	16
3.6	Schema Composition and Reuse	18
4	Validation Rules	20
4.1	Two Classes of Validation	20
4.2	Writing Semantic Validators	21
4.3	Parse-Time Validators	22
4.4	Plan-Time Validators	24

4.5	Error Reporting Format	26
4.6	Example: Validator in the Manifest	27
5	Lowering	28
5.1	The Lowering Contract	28
5.2	Purity Requirement	29
5.3	What Lowering Must Preserve	29
5.4	What Lowering May Add	30
5.5	The Compiler Binary Interface	31
5.6	Example: Lowering <code>approval-request</code>	32
5.7	Example: Compiler Pseudocode	33
5.8	Testing Your Compiler	37
6	Projections	38
6.1	What Projections Are	38
6.2	The Trace Event Stream	38
6.3	Writing a Projection	39
6.4	Use Cases	40
6.5	The Projection Binary Interface	40
6.6	Example: Audit Log Projection	41
6.7	Example: Projection Pseudocode	42
6.8	Projection Declaration in Manifest	44
6.9	Streaming Projections	44
7	Testing Contracts	46
7.1	Kit Acceptance Test Structure	46
7.2	Test Fixture Format	47
7.3	Running Kit Tests	48
7.4	What to Test	49
7.5	Example Test Fixture for Validation	50
7.6	Automating Tests with a Makefile	51
7.7	Continuous Integration	52
7.8	Versioning Test Fixtures	52

8	Publishing	53
8.1	Local-First Model	53
8.2	Referencing a Kit from a Runbook	53
8.3	Kit Discovery	54
8.4	Packaging	55
8.5	Versioning and Pinning	55
8.6	Distribution Options	56
8.7	Updating a Kit	57
8.8	Example: Publishing Checklist	58
9	Reference	59
9.1	Kit Manifest Field Reference	59
9.2	Kit Compiler Binary Interface	61
9.2.1	Invocation	61
9.2.2	Arguments	61
9.2.3	Input	61
9.2.4	Output	61
9.2.5	Exit Codes	62
9.3	Validator Binary Interface	62
9.3.1	Invocation	62
9.3.2	Input	62
9.3.3	Output	63
9.3.4	Exit Codes	63
9.4	Projection Binary Interface	64
9.4.1	Invocation	64
9.4.2	Input	64
9.4.3	Output	64
9.4.4	Exit Codes	64
9.5	Environment Variables	65
9.6	Error Codes and Meanings	65
9.6.1	Compiler Exit Codes	65
9.6.2	Validator Exit Codes	65
9.6.3	Projection Exit Codes	65
9.7	Trace Event Schema Reference	66
9.7.1	Event Types	66
9.8	Common Pitfalls	66

Chapter 1

Introduction

1.1 What Are Domain Kits?

The gert v2 runtime is deliberately domain-agnostic. It executes steps, enforces governance, captures evidence, and writes traces—but it carries no opinion about *what kind of work* a runbook describes. Whether you are orchestrating a compliance audit, provisioning infrastructure, migrating databases, or automating incident response, the gert core executes the same primitive operations: run CLI commands, invoke tools, pause for manual approval, branch on conditions, iterate over lists.

This domain neutrality is not an omission—it is the core design principle. A stable, long-lived runtime cannot accommodate an ever-expanding union of domain-specific vocabularies without becoming brittle and unmaintainable.

Domain Kits are the mechanism by which domain-specific semantics enter gert without contaminating the kernel. A Domain Kit is a compile-time semantic layer that sits *above* the gert core runtime. It provides:

1. **Authoring schemas** — domain-specific YAML vocabulary (custom step types, field structures, validation constraints) that feels natural for authors working in a particular domain.
2. **Validation rules** — semantic validators that enforce domain-specific invariants (“every approval must have a fallback timeout,” “cleanup must follow provision”).
3. **Lowering (compilation)** — a pure transformation that compiles Kit-specific authoring syntax into core gert primitives before execution.
4. **Projections** — read-only transformations over run traces that produce domain-specific reports (audit logs, timeline views, evidence bundles).

5. **Testing contracts** — fixture runbooks and expected outputs that define what “correct” means for this domain.

Domain Kits are *not* runtime extensions. They do not add new capabilities to the gert engine, register tools, or subscribe to events. A Kit operates entirely at **author time** and **compile time**. By the time the gert runtime starts executing a runbook, all Kit-specific syntax has been lowered into core primitives.

1.2 Why Domain Kits Exist

Without Domain Kits, domain-specific semantics have two bad places to live:

Option 1: Leak into the core schema. Every new domain (compliance, workflow automation, data migration, system provisioning) adds fields and step types to the core. The schema becomes a growing union type. The parser grows mode flags. The runtime accumulates conditionals. The kernel loses its stability guarantee. This is the path to a bloated, unmaintainable system.

Option 2: Push into extensions. Extensions add runtime capabilities (tools, event subscriptions, policy enforcement). If we also use them for authoring semantics, we conflate two fundamentally different concerns. An extension that contributes validators, authoring schemas, and projections is doing work that has nothing to do with runtime capabilities. Worse, it forces authoring-layer concerns into a runtime architecture, making them heavier than they need to be.

Domain Kits solve this by providing a clean home for domain semantics. They preserve the kernel’s stability while allowing unlimited domain-specific expressiveness. The gert core schema never changes when a new domain is added—the Kit compiles domain vocabulary down to core primitives.

1.3 Relationship to Gert Core

Domain Kits **compile down to core primitives**. The gert core is unchanged. A Kit is a higher-level authoring syntax built *on top of* the core runtime, not a modification of it.

The pipeline is:

Kit YAML → Kit Parser → Kit AST → Kit Compiler → Core YAML → Core Parser → ExecutionPlan

Key properties:

- The Kit compiler is a **pure function**: YAML in, YAML out, no side effects.
- The lowered output is a valid runbook/v2 document that uses only core step types (`cli`, `tool`, `manual`, `branch`, `iterate`).
- The runtime **never sees Kit-specific nodes**. Lowering happens before the planner produces an ExecutionPlan.
- The lowered runbook can be inspected, version-controlled, and executed **without the Kit installed**. This is critical for portability and auditability.

1.4 The Three Extension Points

Gert v2 has three distinct extension mechanisms. Each serves a different purpose and operates at a different phase:

Domain Kits (authoring semantics) contribute domain-specific YAML vocabulary, validation rules, lowering logic, and projections. They operate at **author time** and **compile time**. They are pure transformations with no runtime footprint.

Extensions (runtime capabilities) contribute tools, event subscriptions, schema field extensions (`x-*` namespaced fields), policy rules, and provider integrations. They operate at **run time** as out-of-process children of the gert engine.

Tools (effectful actions) perform side-effecting work: binary execution, API calls, MCP tool invocations. They operate at **run time**, invoked on demand by the gert engine when a `tool` step executes.

How Kits differ from Extensions and Tools.

- **Kits are compile-time; Extensions are runtime.** A Kit compiler runs once before execution. An Extension runs throughout execution as a persistent child process.
- **Kits are pure; Extensions and Tools have side effects.** A Kit never invokes binaries, calls APIs, or writes to disk (except for its own compiled output). Extensions and Tools perform effectful work.

- **Kits shape vocabulary; Extensions shape capabilities.** A Kit determines what authors can write. An Extension determines what the runtime can do.

A Kit distribution may *ship alongside* an Extension (e.g., a “compliance pack” that includes both a Domain Kit and an Extension with compliance-specific tools), but they remain architecturally distinct.

1.5 Who Should Read This Guide

This guide is for **Kit developers**—engineers who are building domain-specific authoring layers for gert. It is *not* for runbook authors who are using an existing Kit.

You should read this guide if you:

- Want to create a custom step type for your domain (e.g., `approval-request`, `deploy-canary`, `migrate-table`).
- Need to enforce domain-specific validation rules that cannot be expressed in JSON Schema alone (e.g., “every provision step must be followed by a cleanup step”).
- Want to generate domain-specific reports from gert run traces (e.g., audit logs, compliance evidence bundles, timeline views).
- Are building a reusable authoring framework for a community (e.g., a Kit for Kubernetes deployments, database migrations, or security audits).

Prerequisites. This guide assumes you are comfortable with:

- YAML syntax and structure
- JSON Schema (Draft 2020-12)
- Writing and invoking command-line tools
- Basic compiler concepts (parsing, transformation, lowering)

You do *not* need to know the gert v2 runtime internals. This guide stands alone. However, for deeper understanding of the core execution model, consult the *Gert v2 Design Document* (particularly Chapter 4, Domain Kit Model).

1.6 How This Guide Is Structured

This guide follows the Kit development lifecycle:

Chapter 1: Kit Anatomy — directory structure, manifest format, versioning contract.

Chapter 2: Authoring Schemas — how to define custom step types as YAML overlays on core schemas.

Chapter 3: Validation Rules — writing parse-time and plan-time validators.

Chapter 4: Lowering — writing the Kit compiler that transforms your authoring syntax into core primitives.

Chapter 5: Projections — building read models over run traces.

Chapter 6: Testing Contracts — acceptance tests for your Kit.

Chapter 7: Publishing — packaging and distributing your Kit.

Chapter 8: Reference — complete field reference for manifest and binary interfaces.

Work through the chapters sequentially if you are building a Kit from scratch. Use Chapter 8 as a quick reference once you are familiar with the model.

1.7 Forward Reference

For details on the gert v2 runtime, execution model, governance layer, evidence system, and event model, consult the *Gert v2 Design Document*. In particular:

- **Chapter 4: Domain Kit Model** — the architectural rationale for Kits and their relationship to core primitives.
- **Chapter 5: Runtime Architecture** — the execution loop, state machine, and planner.
- **Chapter 6: Governance and Policy** — allowlists, redaction, approval gates, evidence capture.
- **Chapter 9: Event Model** — trace events, event schema, and event lifecycle.

This guide focuses exclusively on *building* Kits, not on the core runtime that executes them.

Chapter 2

Kit Anatomy

2.1 Directory Structure

A Domain Kit is a directory containing a manifest, a compiler binary, schema artifacts, validators, projections, and test fixtures. The layout is standardized to enable tooling (e.g., `gert kit test`) to discover Kit components without additional configuration.

Canonical structure:

```

gert.compliance/           # Kit root directory
+- gert-kit.yaml           # Kit manifest (REQUIRED)
+- bin/
|   +- gert-kit-compliance # Compiler binary (REQUIRED)
|   +- validate-owner      # Optional: individual validator binary
|   L- project-audit-log   # Optional: individual projection binary
+- schemas/
|   L- compliance-v1.json  # Kit-specific JSON Schema overlay
+- test/
|   +- fixtures/           # Test fixture runbooks
|   |   +- 01-basic-approval.yaml
|   |   +- 02-nested-governance.yaml
|   |   L- 03-invalid-missing-owner.yaml
|   L- expected/           # Expected lowered output and validation results
|       +- 01-basic-approval.lowered.yaml
|       +- 02-nested-governance.lowered.yaml
|       L- 03-invalid-missing-owner.errors.json
L- README.md               # Optional: Kit documentation

```

Required components:

`gert-kit.yaml` — The Kit manifest. Declares identity, compatibility, schema contributions, compiler entry point, validators, and projections. See §2.2.

`bin/<compiler-binary>` — The Kit compiler. A standalone executable that reads Kit YAML on stdin and writes core `runbook/v2` YAML to stdout. Must be named in the manifest under `compiler.executable`.

Optional components:

`schemas/*.json` — JSON Schema overlays that define Kit-specific step types and field structures. Referenced from the manifest under `schema.json-schema`.

`bin/validate-*` — Individual validator binaries. If your validators are bundled into the compiler, this directory may be empty.

`bin/project-*` — Individual projection binaries. These consume trace event streams (NDJSON on stdin) and produce projection output (YAML or JSON on stdout).

`test/fixtures/` — Test runbooks authored with this Kit’s vocabulary. Used by `gert kit test`.

`test/expected/` — Expected lowered output and validation results for test fixtures. Used to assert correctness of the Kit compiler.

2.2 The Kit Manifest

The `gert-kit.yaml` manifest is a YAML document with `apiVersion: kit/v1`. It declares the Kit’s identity, compatibility constraints, schema contributions, compiler location, and optional validators and projections.

Complete example:

```
# gert-kit.yaml - Domain Kit Manifest
apiVersion: kit/v1

meta:
```

```
name: gert.compliance          # Reverse-DNS namespaced identifier
version: "1.2.0"               # Semantic version (MUST be semver)
description: "Compliance and audit-focused runbook authoring kit"
author: "Acme Corp <compliance@acme.example>"
homepage: "https://github.com/acme/gert-kit-compliance"

compatibility:
  gert-core: ">=2.0.0 <3.0.0"  # Semver range for gert core compatibility

schema:
  apiVersion-suffix: "compliance/v1" # Authors use: runbook/v2+compliance/v1
  json-schema: "./schemas/compliance-v1.json"

compiler:
  executable: "./bin/gert-kit-compliance"
  args: ["compile"]             # Optional args passed to compiler
  input-format: yaml
  output-format: yaml

step-types:                    # Domain-specific step types this Kit defines
- name: approval-request
  lowers-to: [manual, tool]    # Core types this compiles into
  description: "Request approval from named approvers with timeout"
- name: evidence-capture
  lowers-to: [manual]
  description: "Capture structured evidence with attestation"

validators:                   # Optional domain-specific validators
- name: compliance/owner-required
  phase: parse                 # parse / plan
  description: "Every compliance runbook must declare an owner in meta.roles"
- name: compliance/evidence-on-sensitive
  phase: plan
  description: "Steps marked sensitive must capture evidence"

projections:                  # Optional read models over trace data
- name: compliance/audit-log
  description: "Audit log view with approver attestations and evidence links"
```

```
- name: compliance/evidence-bundle
  description: "Compliance evidence bundle in JSON format"
```

Required fields:

`apiVersion` — MUST be `kit/v1`.

`meta.name` — Reverse-DNS namespaced identifier (e.g., `gert.compliance`, `com.acme.workflow`).

`meta.version` — Semantic version string (MUST follow semver).

`meta.description` — Human-readable summary of the Kit's purpose.

`compiler.executable` — Path (relative to Kit root) to the compiler binary.

Optional but recommended fields:

`compatibility.gert-core` — Semver range specifying which gert core versions this Kit is compatible with. If omitted, the Kit claims compatibility with all versions (not recommended).

`schema.apiVersion-suffix` — The suffix appended to `runbook/v2+` in runbooks authored with this Kit. Example: `compliance/v1` results in `apiVersion: runbook/v2+compliance/v1`.

`schema.json-schema` — Path to the Kit's JSON Schema overlay.

`step-types` — List of Kit-defined step types with metadata about what core types they lower to.

2.3 Kit Versioning Contract

Domain Kits follow semantic versioning (semver). The version in `meta.version` has the following contract:

Major version bump (X.0.0): Breaking changes to the authoring schema. Examples:

- Removing a step type
- Renaming a required field

- Changing field semantics in a non-backward-compatible way
- Changing the `apiVersion-suffix` (e.g., `compliance/v1` → `compliance/v2`)

Minor version bump (1.X.0): Additive, backward-compatible changes. Examples:

- Adding a new step type
- Adding optional fields to an existing step type
- Adding a new validator or projection
- Improving error messages without changing schema

Patch version bump (1.2.X): Bug fixes with no schema changes. Examples:

- Fixing a lowering bug that produces incorrect core YAML
- Fixing a validator that incorrectly rejects valid input
- Fixing a projection that produces malformed output

Pinning in runbooks: Runbook authors can pin to a specific Kit version in the runbook metadata:

```
apiVersion: runbook/v2+compliance/v1
meta:
  kit:
    name: gert.compliance
    version: "1.2.0"           # Exact version pin
  # OR:
  kit:
    name: gert.compliance
    version: "^1.2.0"         # Semver range (any 1.x >= 1.2.0)
```

If the installed Kit version does not satisfy the pin, `gert validate` rejects the runbook with a structured error.

2.4 Example Kit: gert.compliance

To make this concrete, here is the complete file tree for a minimal compliance Kit:

```
gert.compliance/
+- gert-kit.yaml
+- bin/
|   L- gert-kit-compliance      # Go binary (compiled from cmd/compiler/main.go)
+- schemas/
|   L- compliance-v1.json      # JSON Schema overlay
+- test/
|   +- fixtures/
|   |   L- basic-approval.yaml  # Fixture runbook using approval-request
|   |   L- expected/
|   |       L- basic-approval.lowered.yaml # Expected lowered output
L- README.md
```

The `gert-kit-compliance` binary is invoked by `gert` when processing a runbook with `apiVersion: runbook/v2+compliance/v1`. It reads Kit YAML on stdin, validates it against the Kit schema, and writes lowered `runbook/v2` YAML to stdout.

In the next chapter, we will define the Kit schema that describes the `approval-request` step type.

Chapter 3

Authoring Schemas

3.1 What Kit Schemas Are

A Kit schema is a JSON Schema (Draft 2020-12) document that defines domain-specific step types as *overlays* on top of the gert core schema. The core schema defines the base step types (`cli`, `tool`, `manual`, `branch`, `iterate`) and their required fields. A Kit schema adds new step types with domain-specific field structures.

Key principles:

- A Kit schema is **additive**. It does not modify or remove core step types. It adds new types that will be lowered into core types.
- A Kit schema is **validated at parse time**. When a runbook declares `apiVersion: runbook/v2+compliance/v1`, the gert parser loads the Kit schema and validates the runbook against it.
- A Kit schema is **domain-specific**. It should feel natural for authors working in that domain. A compliance Kit might define `approval-request` and `evidence-capture`; a workflow Kit might define `select-priority` and `escalate`.

3.2 Defining a Custom Step Type

A custom step type is defined as a JSON Schema object that extends the base step schema with additional required and optional fields.

Schema overlay pattern:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://gert.dev/schemas/kits/compliance-v1.json",
  "title": "Compliance Kit v1 Schema",
  "description": "Compliance-focused authoring vocabulary for gert",

  "definitions": {
    "approval-request": {
      "type": "object",
      "required": ["id", "type", "approvers"],
      "properties": {
        "id": { "type": "string" },
        "type": { "const": "approval-request" },
        "description": { "type": "string" },
        "approvers": {
          "type": "array",
          "items": { "type": "string" },
          "minItems": 1,
          "description": "List of user identifiers who can approve this step"
        }
      },
    },
    "timeout": {
      "type": "string",
      "pattern": "^[0-9]+(s|m|h|d)$",
      "description": "Max wait time (e.g., '2h', '30m'). Auto-reject on  
↔ timeout."
    },
    "evidence-required": {
      "type": "boolean",
      "default": false,
      "description": "If true, approver must attach evidence"
    },
    "x-kit:metadata": {
      "type": "object",
      "description": "Kit-specific metadata preserved through lowering"
    }
  },
  "additionalProperties": false
}
```

```
  },  
  
  "oneOf": [  
    { "$ref": "#/definitions/approval-request" }  
  ]  
}
```

Field conventions:

id — REQUIRED. Unique step identifier within the runbook.

type — REQUIRED. Step type discriminator. Use **"const"** to enforce exact match.

description — Optional but recommended. Human-readable step summary.

Domain fields — Add required and optional fields specific to your domain (e.g., **approvers**, **timeout**, **evidence-required**).

x-kit:* — Prefix for Kit-defined metadata fields. Use this namespace for fields that should be preserved through lowering but are not part of the core schema.

3.3 Field Extension Conventions

Required vs. optional fields: Use **"required"** to declare fields that **MUST** be present for semantic correctness. Example: **approvers** is required for **approval-request** because an approval without approvers is meaningless.

Optional fields should have sensible defaults or be truly optional. Document the default behavior in the schema **description**.

Type constraints: Use JSON Schema **type**, **pattern**, **minItems**, **enum**, etc., to enforce structural correctness. Example:

```
"timeout": {  
  "type": "string",  
  "pattern": "^[0-9]+(s|m|h|d)$",  
  "description": "Duration with unit suffix (s=seconds, m=minutes, h=hours,  
    ↪ d=days)"  
}
```

The x-kit: annotation prefix: Fields prefixed with `x-kit:` are Kit-specific metadata that will be preserved through lowering but are not interpreted by the gert core. Use this for:

- Hints to the lowering compiler (e.g., `x-kit:compile-mode: strict`)
- Metadata for projections (e.g., `x-kit:audit-category: sensitive`)
- Domain-specific annotations (e.g., `x-kit:compliance-framework: SOC2`)

The gert core ignores `x-kit:*` fields. Your Kit compiler can read them from the AST and use them to inform lowering behavior.

3.4 Validation at Parse Time vs. Plan Time

There are two phases of validation in the gert pipeline:

Parse-time validation runs when `gert validate` is invoked or when a runbook is loaded.

The parser validates the runbook YAML against the Kit schema (structural validation).

This catches syntax errors, missing required fields, type mismatches, etc.

Plan-time validation runs after lowering, when the planner is about to produce an `ExecutionPlan`. This is where semantic validators (§4) run. Plan-time validators can see the full lowered runbook and enforce cross-step invariants (e.g., “cleanup must follow provision”).

What belongs in the schema (parse-time):

- Field presence (required vs. optional)
- Field types (string, number, array, object)
- Format constraints (patterns, enums, ranges)
- Per-step structural correctness

What belongs in semantic validators (plan-time):

- Cross-step invariants (“every X must be followed by Y”)
- Business rules (“timeout must not exceed SLA”)
- Context-dependent validation (“if field A is set, field B is required”)

Use JSON Schema for structure; use semantic validators for semantics.

3.5 Example: evidence-capture Step Type

Let’s define a second custom step type: `evidence-capture`. This step pauses execution and prompts a human to provide structured evidence (text, file upload, or both).

```
{
  "definitions": {
    "evidence-capture": {
      "type": "object",
      "required": ["id", "type", "prompt"],
      "properties": {
        "id": { "type": "string" },
        "type": { "const": "evidence-capture" },
        "description": { "type": "string" },
        "prompt": {
          "type": "string",
          "description": "Instruction text shown to the operator"
        },
      },
    },
    "evidence-type": {
      "type": "string",
      "enum": ["text", "attachment", "both"],
      "default": "text",
      "description": "What kind of evidence to collect"
    },
    "required-attestation": {
      "type": "boolean",
      "default": true,
      "description": "If true, operator must attest evidence is accurate"
    }
  }
}
```

```
    },
    "x-kit:audit-tag": {
      "type": "string",
      "description": "Tag for grouping evidence in audit reports"
    }
  },
  "additionalProperties": false
}
}
```

Authoring example: A runbook author using the compliance Kit might write:

```
apiVersion: runbook/v2+compliance/v1
meta:
  title: "Quarterly access review"
  kit:
    name: gert.compliance
    version: "~1.0.0"

steps:
- id: request-approval
  type: approval-request
  description: "Get approval from security team"
  approvers:
    - alice@acme.example
    - bob@acme.example
  timeout: 2h
  evidence-required: true

- id: capture-review-results
  type: evidence-capture
  description: "Upload access review spreadsheet"
  prompt: "Attach the completed access review CSV file"
  evidence-type: attachment
  x-kit:audit-tag: "access-review-q1-2025"
```

This Kit-authored runbook will be validated against the compliance Kit schema at parse time, then lowered into core step types before execution.

3.6 Schema Composition and Reuse

For complex Kits with many step types, use JSON Schema `$ref` and `definitions` to avoid repetition:

```
{
  "definitions": {
    "timeout-field": {
      "type": "string",
      "pattern": "^[0-9]+(s|m|h|d)$"
    },
    "user-list": {
      "type": "array",
      "items": { "type": "string" },
      "minItems": 1
    },
    "approval-request": {
      "type": "object",
      "required": ["id", "type", "approvers"],
      "properties": {
        "id": { "type": "string" },
        "type": { "const": "approval-request" },
        "approvers": { "$ref": "#/definitions/user-list" },
        "timeout": { "$ref": "#/definitions/timeout-field" }
      }
    },
    "escalate": {
      "type": "object",
      "required": ["id", "type", "escalate-to"],
      "properties": {
        "id": { "type": "string" },
        "type": { "const": "escalate" },
        "escalate-to": { "$ref": "#/definitions/user-list" },
        "timeout": { "$ref": "#/definitions/timeout-field" }
      }
    }
  }
}
```

```
}  
  }  
    }  
  }
```

This reduces duplication and ensures consistency across step types.

Chapter 4

Validation Rules

4.1 Two Classes of Validation

Validation in the Domain Kit model operates at two levels:

Structural validation (schema) is performed by JSON Schema at parse time. It enforces field presence, types, formats, and per-step correctness. This is covered in Chapter 3.

Semantic validation (business rules) is performed by Kit-specific validators at parse time or plan time. These are executable binaries that enforce domain-specific invariants that cannot be expressed in JSON Schema alone.

When to use semantic validators:

- **Cross-step invariants:** “Every provision step must be followed by a cleanup step.”
- **Contextual constraints:** “If `timeout` exceeds SLA, emit a warning.”
- **Domain-specific business rules:** “Every runbook in the compliance Kit must declare an owner in `meta.roles`.”
- **Complex conditions:** “If step A sets variable X, step B (if present) must reference X.”

If the rule can be expressed in JSON Schema, use JSON Schema. Semantic validators are for everything else.

4.2 Writing Semantic Validators

A semantic validator is a standalone executable that reads the parsed runbook (or the lowered runbook) on stdin and writes a list of validation errors and warnings to stdout.

Input contract: The validator receives the runbook as YAML on stdin. For parse-time validators, this is the Kit-authored runbook. For plan-time validators, this is the lowered runbook/v2 document.

Output contract: The validator writes a JSON array of validation messages to stdout. Each message has:

file — File path (or "<stdin>" if from a pipe).

line — Line number where the issue occurs (1-indexed).

column — Column number (1-indexed, optional).

severity — "error" or "warning".

message — Human-readable description of the issue.

rule — Validator rule identifier (e.g., "compliance/owner-required").

Example output:

```
[
  {
    "file": "runbook.yaml",
    "line": 3,
    "column": 1,
    "severity": "error",
    "message": "Compliance runbooks must declare an owner in meta.roles.owner",
    "rule": "compliance/owner-required"
  },
  {
    "file": "runbook.yaml",
    "line": 15,
    "severity": "warning",
```

```
    "message": "Timeout '4h' exceeds recommended SLA of 2h",  
    "rule": "compliance/timeout-sla"  
  }  
]
```

If validation passes, the validator writes an empty array `[]` to stdout.

Exit code contract:

- 0 — Validation passed (no errors).
- 1 — Validation failed (at least one error).
- 2+ — Internal validator error (e.g., malformed input, panic).

Warnings do not cause a non-zero exit code. Only errors cause exit code 1.

4.3 Parse-Time Validators

Parse-time validators run when `gert validate` is called, *before* lowering. They see the Kit-authored runbook and can enforce authoring-layer invariants.

Use cases:

- Enforce that required metadata fields are present (e.g., `meta.roles.owner`).
- Validate that step IDs follow naming conventions (e.g., no spaces, kebab-case).
- Check that Kit-specific fields are used correctly (e.g., `approvers` list is not empty).

Example: owner-required validator (pseudocode):

```
package main  
  
import (  
    "encoding/json"  
    "fmt"  
    "gopkg.in/yaml.v3"  
    "os"
```

```
)

type ValidationMessage struct {
    File      string `json:"file"`
    Line      int    `json:"line"`
    Severity  string `json:"severity"`
    Message   string `json:"message"`
    Rule      string `json:"rule"`
}

func main() {
    var runbook map[string]interface{}
    yaml.NewDecoder(os.Stdin).Decode(&runbook)

    var messages []ValidationMessage

    meta, ok := runbook["meta"].(map[string]interface{})
    if !ok || meta["roles"] == nil {
        messages = append(messages, ValidationMessage{
            File:      "<stdin>",
            Line:      1,
            Severity:  "error",
            Message:   "Compliance runbooks must declare meta.roles.owner",
            Rule:      "compliance/owner-required",
        })
    } else {
        roles := meta["roles"].(map[string]interface{})
        if roles["owner"] == nil {
            messages = append(messages, ValidationMessage{
                File:      "<stdin>",
                Line:      3,
                Severity:  "error",
                Message:   "meta.roles.owner is required for compliance runbooks",
                Rule:      "compliance/owner-required",
            })
        }
    }
}
```

```

    json.NewEncoder(os.Stdout).Encode(messages)
    if len(messages) > 0 && hasError(messages) {
        os.Exit(1)
    }
}

func hasError(messages []ValidationMessage) bool {
    for _, m := range messages {
        if m.Severity == "error" {
            return true
        }
    }
    return false
}

```

4.4 Plan-Time Validators

Plan-time validators run *after* lowering, when the planner is about to produce an Execution-Plan. They see the lowered `runbook/v2` document (core primitives only) and can enforce cross-step invariants.

Use cases:

- Enforce step ordering rules (e.g., “cleanup must follow provision”).
- Validate that lowered steps have required governance fields populated.
- Check that synthesized variables are referenced correctly.

Example: cleanup-follows-provision validator: This validator checks that for every step with ID matching `provision-*`, there exists a subsequent step with ID matching `cleanup-*`.

```

func validateCleanupFollowsProvision(runbook map[string]interface{})
↳ []ValidationMessage {
    steps := runbook["steps"].([]interface{})
    var messages []ValidationMessage

```

```

provisionIndices := make(map[string]int)

// Find all provision steps
for i, step := range steps {
    s := step.(map[string]interface{})
    id := s["id"].(string)
    if strings.HasPrefix(id, "provision-") {
        provisionIndices[id] = i
    }
}

// Check that each provision has a corresponding cleanup after it
for provID, provIndex := range provisionIndices {
    cleanupID := strings.Replace(provID, "provision-", "cleanup-", 1)
    found := false
    for i := provIndex + 1; i < len(steps); i++ {
        s := steps[i].(map[string]interface{})
        if s["id"].(string) == cleanupID {
            found = true
            break
        }
    }
    if !found {
        messages = append(messages, ValidationMessage{
            File:    "<stdin>",
            Line:    provIndex + 5, // Approximate line number
            Severity: "error",
            Message: fmt.Sprintf("Step '%s' requires a '%s' step after it", provID,
                ↪ cleanupID),
            Rule:    "compliance/cleanup-follows-provision",
        })
    }
}

return messages
}

```

4.5 Error Reporting Format

Validation messages **MUST** be structured JSON to enable tooling (IDEs, CI systems) to parse and display them.

Field reference:

file (string, required) — File path. Use "<stdin>" if the runbook was piped in.

line (integer, required) — Line number (1-indexed).

column (integer, optional) — Column number (1-indexed).

severity (string, required) — **"error"** or **"warning"**. Errors block execution; warnings do not.

message (string, required) — Human-readable description. Should be actionable (tell the author what to fix).

rule (string, required) — Validator rule ID (e.g., **"compliance/owner-required"**). Namespaced with Kit name.

Severity guidelines:

- Use **"error"** for violations that would cause incorrect behavior at runtime or violate hard requirements (e.g., missing required field, invariant violation).
- Use **"warning"** for stylistic issues, recommendations, or soft constraints (e.g., “timeout exceeds recommended SLA”).

Message guidelines:

- Be specific: “Step 'provision-db' requires a 'cleanup-db' step after it” is better than “Invalid step order.”
- Be actionable: “Add meta.roles.owner field” is better than “Owner missing.”
- Reference the rule ID in the message if it helps understanding.

4.6 Example: Validator in the Manifest

Validators are declared in the Kit manifest under the `validators` key:

```
validators:
- name: compliance/owner-required
  phase: parse           # parse / plan
  description: "Every compliance runbook must declare an owner in meta.roles"
- name: compliance/cleanup-follows-provision
  phase: plan
  description: "Every provision step must be followed by a cleanup step"
- name: compliance/timeout-sla
  phase: parse
  description: "Warns if timeout exceeds recommended SLA"
```

Validator invocation: Gert invokes validators by:

1. Reading the Kit manifest to discover declared validators.
2. Executing the Kit compiler with a `-validate` flag (or a separate validator binary if specified).
3. Passing the runbook on stdin.
4. Collecting validation messages from stdout.
5. Aggregating messages from all validators and reporting them to the user.

If your Kit bundles validators into the compiler binary, handle the `-validate` subcommand:

```
./bin/gert-kit-compliance compile < runbook.yaml > lowered.yaml
./bin/gert-kit-compliance validate < runbook.yaml > errors.json
```

If you prefer separate binaries per validator, structure them as:

```
./bin/validate-owner-required < runbook.yaml > errors.json
./bin/validate-cleanup-follows-provision < lowered.yaml > errors.json
```

Chapter 5

Lowering

5.1 The Lowering Contract

Lowering is the process of compiling Kit-authored runbook YAML into core `runbook/v2` YAML. The Kit compiler is a pure function:

Kit YAML (stdin) → Kit Compiler → Core YAML (stdout)

Input: A runbook with `apiVersion: runbook/v2+<kit-suffix>` that uses Kit-specific step types (e.g., `approval-request`, `evidence-capture`).

Output: A valid `runbook/v2` document that uses only core step types (`cli`, `tool`, `manual`, `branch`, `iterate`) and core fields.

Key properties:

- **Purity:** The compiler has no side effects. It does not invoke binaries, call APIs, write to disk (except stdout), or access the network.
- **Determinism:** The same input always produces the same output. This is critical for testing and reproducibility.
- **Correctness:** The lowered runbook must be semantically equivalent to the Kit-authored runbook. Lowering can add steps (synthesized steps) but must preserve the author's intent.

5.2 Purity Requirement

The Kit compiler **MUST** be a pure transformation. This means:

- No filesystem reads (except stdin).
- No filesystem writes (except stdout).
- No network requests.
- No subprocess invocations.
- No environment variable dependencies (except for optional debug flags).
- No non-deterministic operations (random numbers, timestamps, etc.).

Why purity matters:

1. **Testability:** Pure functions are easy to test. You provide input, assert output, done.
2. **Reproducibility:** The same runbook lowered today and lowered tomorrow produces identical output. This is essential for version control and CI systems.
3. **Auditability:** Lowering is transparent. You can inspect the lowered output and trace it back to the Kit input line-by-line.
4. **Portability:** A pure compiler can run anywhere (local laptop, CI runner, airgapped environment) without external dependencies.

If your Kit needs runtime state (e.g., fetching user lists from an API), that belongs in a **Tool** or **Extension**, not in the compiler.

5.3 What Lowering Must Preserve

The lowering **MUST** preserve the following:

Step order — Steps execute in the same order as authored (unless the author explicitly uses `branch` or `iterate`).

Variable bindings — If the Kit-authored runbook sets a variable `X`, the lowered runbook must set `X` with the same value.

Governance hooks — If a Kit step specifies governance fields (e.g., `approval`, `allow`, `redact`), those must be preserved in the lowered core steps.

Evidence slots — If a Kit step captures evidence, the lowered core step must have the corresponding evidence field.

Step metadata — The lowered step should preserve `id`, `description`, and any other metadata from the Kit step.

5.4 What Lowering May Add

The lowering MAY add the following:

Synthesized steps — Steps that were not authored but are necessary for correctness.
Example: a Kit might synthesize a setup step before the first user-authored step.

Variable injections — The compiler may inject variables into the runbook’s `vars` section (e.g., Kit version, lowering timestamp).

Pre/post hooks — The compiler may wrap user steps with pre-step or post-step hooks (implemented as additional core steps).

Governance field population — If a Kit step implies governance (e.g., `approval-request` implies `approval: manual`), the compiler populates those fields in the lowered output.

Synthesized step ID conventions: If your compiler synthesizes steps, use a naming convention that makes it clear they are synthetic:

- Prefix with `__kit__` (e.g., `__kit__setup`).
- Or suffix with `__synthetic` (e.g., `init__synthetic`).

This helps debuggers and trace viewers distinguish authored steps from synthesized steps.

5.5 The Compiler Binary Interface

Invocation:

```
<kit-compiler-binary> compile < input.yaml > output.yaml  
echo $?    # Exit code: 0 = success, 1 = error
```

Stdin: Kit-authored runbook YAML.

Stdout: Lowered `runbook/v2` YAML (if successful) OR error messages in JSON format (if failed).

Stderr: Diagnostic messages, debug logs (optional, not parsed by gert).

Exit codes:

- 0 — Lowering succeeded. Stdout contains valid `runbook/v2` YAML.
- 1 — Lowering failed. Stdout contains a JSON array of error messages (same format as validator output, §4.5).
- 2+ — Internal compiler error (panic, unexpected failure). Stderr may contain a stack trace.

Error output format: If lowering fails (exit code 1), write a JSON array of error messages to stdout:

```
[  
  {  
    "file": "runbook.yaml",  
    "line": 12,  
    "severity": "error",  
    "message": "approval-request step requires at least one approver",  
    "rule": "compliance/lowering"  
  }  
]
```

5.6 Example: Lowering approval-request

Let's lower a Kit-authored `approval-request` step into a core `manual` step with governance fields populated.

Kit input:

```
apiVersion: runbook/v2+compliance/v1
meta:
  title: "Deploy to production"

steps:
  - id: approve-deploy
    type: approval-request
    description: "Get approval from ops team"
    approvers:
      - alice@acme.example
      - bob@acme.example
    timeout: 2h
    evidence-required: true
```

Lowered output:

```
apiVersion: runbook/v2
meta:
  title: "Deploy to production"
  x-kit:lowered-from:
    kit: gert.compliance
    version: "1.2.0"

steps:
  - id: approve-deploy
    type: manual
    description: "Get approval from ops team"
    prompt: |
      Approval required from:
      - alice@acme.example
      - bob@acme.example
```

```
Timeout: 2h
Evidence required: Yes

approval:
  required: true
  approvers:
    - alice@acme.example
    - bob@acme.example
  timeout: 7200 # 2h in seconds

evidence:
  required: true
  type: text
  prompt: "Provide evidence for this approval (required)"

on-timeout:
  action: reject
  message: "Approval timed out after 2h"
```

Transformation notes:

- The Kit step type `approval-request` is lowered to core type `manual`.
- Kit fields (`approvers`, `timeout`, `evidence-required`) are mapped to core governance fields (`approval`, `evidence`, `on-timeout`).
- The compiler synthesized a `prompt` field to guide the operator.
- Metadata was added to `meta.x-kit:lowered-from` to record which Kit and version performed the lowering.

5.7 Example: Compiler Pseudocode

Here is pseudocode for a minimal Kit compiler (Go):

```
package main
```

```
import (
    "encoding/json"
    "fmt"
    "gopkg.in/yaml.v3"
    "os"
)

func main() {
    var runbook map[string]interface{}
    if err := yaml.NewDecoder(os.Stdin).Decode(&runbook); err != nil {
        reportError("Failed to parse input YAML", 1)
        os.Exit(1)
    }

    lowered, err := lowerRunbook(runbook)
    if err != nil {
        reportError(err.Error(), 1)
        os.Exit(1)
    }

    yaml.NewEncoder(os.Stdout).Encode(lowered)
}

func lowerRunbook(rb map[string]interface{}) (map[string]interface{}, error) {
    // Change apiVersion from runbook/v2+compliance/v1 to runbook/v2
    rb["apiVersion"] = "runbook/v2"

    // Add lowering metadata
    meta := rb["meta"].(map[string]interface{})
    meta["x-kit:lowered-from"] = map[string]string{
        "kit":      "gert.compliance",
        "version": "1.2.0",
    }
}

// Lower each step
steps := rb["steps"].([]interface{})
for i, step := range steps {
    s := step.(map[string]interface{})
```

```

    loweredStep, err := lowerStep(s)
    if err != nil {
        return nil, fmt.Errorf("step %d: %w", i, err)
    }
    steps[i] = loweredStep
}

return rb, nil
}

func lowerStep(step map[string]interface{}) (map[string]interface{}, error) {
    stepType := step["type"].(string)

    switch stepType {
    case "approval-request":
        return lowerApprovalRequest(step)
    case "evidence-capture":
        return lowerEvidenceCapture(step)
    default:
        // Core step types pass through unchanged
        return step, nil
    }
}

func lowerApprovalRequest(step map[string]interface{}) (map[string]interface{},
→ error) {
    approvers := step["approvers"].([]interface{})
    if len(approvers) == 0 {
        return nil, fmt.Errorf("approval-request requires at least one approver")
    }

    timeout := step["timeout"].(string) // e.g., "2h"
    timeoutSec := parseTimeout(timeout) // Convert to seconds

    lowered := map[string]interface{}{
        "id":      step["id"],
        "type":    "manual",

```

```

    "description": step["description"],
    "prompt":      buildApprovalPrompt(step),
    "approval": map[string]interface{}{
        "required": true,
        "approvers": approvers,
        "timeout":  timeoutSec,
    },
}

if step["evidence-required"] == true {
    lowered["evidence"] = map[string]interface{}{
        "required": true,
        "type":      "text",
        "prompt":    "Provide evidence for this approval (required)",
    }
}

return lowered, nil
}

func buildApprovalPrompt(step map[string]interface{}) string {
    approvers := step["approvers"].([]interface{})
    timeout := step["timeout"].(string)
    var prompt string
    prompt += "Approval required from:\n"
    for _, a := range approvers {
        prompt += fmt.Sprintf("  - %s\n", a)
    }
    prompt += fmt.Sprintf("\nTimeout: %s\n", timeout)
    if step["evidence-required"] == true {
        prompt += "Evidence required: Yes\n"
    }
    return prompt
}

func parseTimeout(t string) int {
    // Parse "2h" → 7200 seconds (simplified; real impl would handle all units)
    // ...

```

```
    return 7200
}

func reportError(msg string, line int) {
    errors := []map[string]interface{}{
        {
            "file":    "<stdin>",
            "line":   line,
            "severity": "error",
            "message": msg,
            "rule":    "compliance/lowering",
        },
    }
    json.NewEncoder(os.Stdout).Encode(errors)
}
```

5.8 Testing Your Compiler

The compiler **MUST** be tested with fixture runbooks and expected outputs. See Chapter 7 for the full testing workflow. At a minimum:

1. Create a fixture runbook: `test/fixtures/basic-approval.yaml`
2. Run your compiler: `cat basic-approval.yaml | ./bin/gert-kit-compliance compile > output.yaml`
3. Assert that `output.yaml` matches the expected lowered form.
4. Validate the lowered output against the core `runbook/v2` schema.

Automate this with a test script or a `Makefile` target.

Chapter 6

Projections

6.1 What Projections Are

A **projection** is a read-only transformation over a gert run trace. It consumes the trace event stream (NDJSON format) produced during runbook execution and produces a domain-specific view: audit logs, timeline summaries, evidence bundles, compliance reports, etc.

Projections are:

- **Read-only** — They never modify the trace. They only derive views from it.
- **Post-hoc** — They run *after* execution completes (or during execution as a streaming consumer).
- **Domain-specific** — Different Kits produce different projections. A compliance Kit might project an audit log; a workflow Kit might project a timeline view.
- **Pure transformations** — Like the compiler, projections have no side effects (no network, no filesystem writes except stdout).

6.2 The Trace Event Stream

Gert emits a sequence of trace events during execution. Each event is a JSON object representing a state transition:

`run.started` — Execution began.

`step.started` — A step began executing.

`step.completed` — A step finished successfully.

`step.failed` — A step failed.

`evidence.captured` — Evidence was collected.

`approval.requested` — Approval gate was reached.

`approval.granted` — Approval was granted.

`approval.rejected` — Approval was denied.

`variable.set` — A variable was bound.

`run.completed` — Execution finished.

Event format: Each event is a JSON object with:

`event` — Event type (e.g., `"step.started"`).

`timestamp` — ISO 8601 timestamp (UTC).

`run_id` — Unique run identifier.

`step_id` — Step ID (for step-level events).

`payload` — Event-specific data (varies by event type).

Example events:

```
{ "event": "run.started", "timestamp": "2025-01-15T10:30:00Z", "run_id": "r-abc123", "payload": {} }
{ "event": "step.started", "timestamp": "2025-01-15T10:30:01Z", "run_id": "r-abc123", "step_id": "approve-c"
{ "event": "evidence.captured", "timestamp": "2025-01-15T10:32:00Z", "run_id": "r-abc123", "step_id": "app
  → deployment checklist. All items OK.", "sha256": "e3b0c..." } }
{ "event": "approval.granted", "timestamp": "2025-01-15T10:32:05Z", "run_id": "r-abc123", "step_id": "appro
{ "event": "step.completed", "timestamp": "2025-01-15T10:32:06Z", "run_id": "r-abc123", "step_id": "approve
{ "event": "run.completed", "timestamp": "2025-01-15T10:35:00Z", "run_id": "r-abc123", "payload": { "status":
```

6.3 Writing a Projection

A projection is a standalone executable that reads NDJSON events on stdin and writes the projection output to stdout.

Input: NDJSON stream of trace events (one JSON object per line).

Output: Projection-specific format (JSON, YAML, Markdown, CSV, etc.).

Exit code:

0 — Projection succeeded.

1+ — Projection failed (malformed input, internal error).

6.4 Use Cases

Audit logs: Extract all approval events, evidence captures, and governance checks into a chronological audit log.

Summary reports: Produce a high-level summary: which steps ran, how long they took, which approvers were involved, what evidence was captured.

Compliance evidence bundles: Extract all evidence items (text, attachments) and group them by governance primitive (approval, attestation, redaction). Output as a structured JSON document for compliance archival.

Timeline views: Render a visual timeline of execution (Markdown, HTML, or JSON for frontend rendering).

Metrics: Extract duration metrics, step counts, approval latencies for dashboards.

6.5 The Projection Binary Interface

Invocation:

```
cat trace.ndjson | ./bin/project-audit-log > audit.json
```

Stdin: NDJSON trace events (one event per line).

Stdout: Projection output (format specified by the projection).

Stderr: Diagnostic messages (optional, not parsed).

6.6 Example: Audit Log Projection

Let's build a projection that extracts all approval and evidence events into an audit log.

Input (trace excerpt):

```
{"event": "step.started", "timestamp": "2025-01-15T10:30:01Z", "run_id": "r-abc123", "step_id": "approve-deploy"},
{"event": "approval.requested", "timestamp": "2025-01-15T10:30:02Z", "run_id": "r-abc123", "step_id": "approve-deploy"},
{"event": "evidence.captured", "timestamp": "2025-01-15T10:32:00Z", "run_id": "r-abc123", "step_id": "approve-deploy"},
{"event": "approval.granted", "timestamp": "2025-01-15T10:32:05Z", "run_id": "r-abc123", "step_id": "approve-deploy"},
{"event": "step.completed", "timestamp": "2025-01-15T10:32:06Z", "run_id": "r-abc123", "step_id": "approve-deploy"}
```

Output (audit log JSON):

```
{
  "run_id": "r-abc123",
  "audit_entries": [
    {
      "timestamp": "2025-01-15T10:30:02Z",
      "step_id": "approve-deploy",
      "event_type": "approval.requested",
      "details": {
        "approvers": ["alice@acme.example", "bob@acme.example"]
      }
    },
    {
      "timestamp": "2025-01-15T10:32:00Z",
      "step_id": "approve-deploy",
      "event_type": "evidence.captured",
      "details": {
        "type": "text",
        "value": "Deployment checklist complete."
      }
    }
  ]
}
```

```
        "timestamp": "2025-01-15T10:32:05Z",
        "step_id": "approve-deploy",
        "event_type": "approval.granted",
        "details": {
            "approver": "alice@acme.example"
        }
    }
]
```

6.7 Example: Projection Pseudocode

Here is pseudocode for the audit log projection (Go):

```
package main

import (
    "bufio"
    "encoding/json"
    "os"
)

type TraceEvent struct {
    Event      string           `json:"event"`
    Timestamp  string           `json:"timestamp"`
    RunID      string           `json:"run_id"`
    StepID     string           `json:"step_id,omitempty"`
    Payload    map[string]interface{} `json:"payload"`
}

type AuditEntry struct {
    Timestamp string           `json:"timestamp"`
    StepID    string           `json:"step_id"`
    EventType string           `json:"event_type"`
    Details   map[string]interface{} `json:"details"`
}
```

```
type AuditLog struct {
    RunID    string    `json:"run_id"`
    Entries []AuditEntry `json:"audit_entries"`
}

func main() {
    scanner := bufio.NewScanner(os.Stdin)
    var log AuditLog
    var runID string

    for scanner.Scan() {
        var evt TraceEvent
        json.Unmarshal(scanner.Bytes(), &evt)

        if runID == "" {
            runID = evt.RunID
            log.RunID = runID
        }

        // Filter: only audit-relevant events
        if isAuditEvent(evt.Event) {
            log.Entries = append(log.Entries, AuditEntry{
                Timestamp: evt.Timestamp,
                StepID:    evt.StepID,
                EventType: evt.Event,
                Details:   evt.Payload,
            })
        }
    }

    json.NewEncoder(os.Stdout).Encode(log)
}

func isAuditEvent(eventType string) bool {
    auditEvents := []string{
        "approval.requested",
        "approval.granted",
        "approval.rejected",
    }
}
```

```
    "evidence.captured",
  }
  for _, e := range auditEvents {
    if eventType == e {
      return true
    }
  }
  return false
}
```

6.8 Projection Declaration in Manifest

Projections are declared in the Kit manifest:

```
projections:
- name: compliance/audit-log
  description: "Audit log with approvals and evidence"
  output-format: json
- name: compliance/evidence-bundle
  description: "Evidence bundle for compliance archival"
  output-format: json
- name: compliance/timeline
  description: "Execution timeline in Markdown"
  output-format: markdown
```

Invocation by gert: Gert can invoke projections automatically after a run completes:

```
gert run runbook.yaml --project compliance/audit-log > audit.json
```

Or manually:

```
cat .gert/traces/r-abc123.ndjson | ./bin/project-audit-log > audit.json
```

6.9 Streaming Projections

Projections can run in **streaming mode**: they consume events as they are emitted during execution, rather than waiting for the run to complete.

Use cases:

- Real-time dashboards (stream events to a web UI).
- Incremental compliance logging (write audit entries as they occur).
- Alerting (emit alerts when specific events occur).

Streaming contract: The projection binary should:

1. Read events from stdin line by line (NDJSON).
2. Process each event and emit incremental output to stdout (optional).
3. Flush stdout after each output (for real-time consumption).
4. Handle incomplete runs gracefully (if stdin closes before `run.completed`, emit partial output).

Gert can invoke streaming projections with:

```
gert run runbook.yaml --stream-to "./bin/project-audit-log" &
```

The projection runs as a child process, consuming events as they are emitted.

Chapter 7

Testing Contracts

7.1 Kit Acceptance Test Structure

A Domain Kit **MUST** include acceptance tests that define what “correct” means for the Kit. These tests verify:

1. **Lowering correctness** — The compiler produces the expected core YAML for Kit-authored runbooks.
2. **Validator behavior** — Validators correctly accept valid input and reject invalid input with actionable error messages.
3. **Projection output** — Projections produce the expected output for a given trace stream.

Test directory structure:

```
test/
+- fixtures/                                # Test input runbooks
|   +- 01-basic-approval.yaml
|   +- 02-nested-governance.yaml
|   +- 03-evidence-capture.yaml
|   L- 04-invalid-missing-owner.yaml
L- expected/                                # Expected outputs
    +- 01-basic-approval.lowered.yaml
    +- 02-nested-governance.lowered.yaml
    +- 03-evidence-capture.lowered.yaml
    L- 04-invalid-missing-owner.errors.json
```

7.2 Test Fixture Format

A test fixture is a Kit-authored runbook that exercises a specific feature or validates a specific error condition.

Example: 01-basic-approval.yaml

```
# Test fixture: basic approval-request lowering
apiVersion: runbook/v2+compliance/v1
meta:
  title: "Basic approval test"
  kit:
    name: gert.compliance
    version: "1.0.0"

steps:
  - id: approve-step
    type: approval-request
    description: "Simple approval with two approvers"
    approvers:
      - alice@acme.example
      - bob@acme.example
    timeout: 1h
```

Expected lowered output: 01-basic-approval.lowered.yaml

```
apiVersion: runbook/v2
meta:
  title: "Basic approval test"
  x-kit:lowered-from:
    kit: gert.compliance
    version: "1.0.0"

steps:
  - id: approve-step
    type: manual
    description: "Simple approval with two approvers"
    prompt: |
```

```
Approval required from:
- alice@acme.example
- bob@acme.example

Timeout: 1h
approval:
  required: true
  approvers:
    - alice@acme.example
    - bob@acme.example
  timeout: 3600
```

7.3 Running Kit Tests

The `gert kit test` command (described here as expected behavior, even if not yet implemented) should:

1. Discover all fixtures in `test/fixtures/`.
2. For each fixture:
 - (a) Run the Kit compiler: `cat fixture.yaml | ./bin/gert-kit-compliance compile > actual.yaml`
 - (b) Compare `actual.yaml` to `expected/*.lowered.yaml`.
 - (c) If the fixture is marked as invalid (e.g., `04-invalid-*.yaml`), assert that the compiler exits with code 1 and produces the expected error JSON.
3. Report results: pass/fail for each fixture.

Manual test invocation: Until `gert kit test` is implemented, run tests manually:

```
# Test 01: basic approval
cat test/fixtures/01-basic-approval.yaml | \
  ./bin/gert-kit-compliance compile > /tmp/actual.yaml

diff -u test/expected/01-basic-approval.lowered.yaml /tmp/actual.yaml
```

```
# Test 04: invalid missing owner (expect error)
cat test/fixtures/04-invalid-missing-owner.yaml | \
  ./bin/gert-kit-compliance compile > /tmp/errors.json
echo $? # Should be 1 (error)

diff -u test/expected/04-invalid-missing-owner.errors.json /tmp/errors.json
```

7.4 What to Test

Lowering correctness:

- Basic step type lowering (one Kit step $\rightarrow$ one or more core steps).
- Field mapping (Kit fields $\rightarrow$ core fields).
- Governance field population (e.g., `approval-request` $\rightarrow$ `approval` field).
- Synthesized steps (if your Kit adds setup/teardown steps).
- Variable injection (if your Kit adds variables to `vars`).

Validator edge cases:

- Valid input passes validation (exit code 0, empty error array).
- Missing required fields produce errors.
- Invalid field values produce errors.
- Cross-step invariants are enforced (e.g., cleanup follows provision).
- Warnings are emitted when appropriate (non-blocking).

Projection output format:

- Projections produce well-formed JSON/YAML/Markdown.
- Projections correctly filter events (only audit-relevant events in audit log).
- Projections handle incomplete traces (if `run.complected` is missing).

7.5 Example Test Fixture for Validation

Fixture: 04-invalid-missing-owner.yaml

```
# Test fixture: missing owner (should fail validation)
apiVersion: runbook/v2+compliance/v1
meta:
  title: "Invalid runbook: missing owner"

steps:
- id: step-1
  type: approval-request
  description: "This should fail because meta.roles.owner is missing"
  approvers:
    - alice@acme.example
  timeout: 1h
```

Expected error output: 04-invalid-missing-owner.errors.json

```
[
  {
    "file": "<stdin>",
    "line": 3,
    "severity": "error",
    "message": "Compliance runbooks must declare an owner in meta.roles.owner",
    "rule": "compliance/owner-required"
  }
]
```

Test invocation:

```
cat test/fixtures/04-invalid-missing-owner.yaml | \
  ./bin/gert-kit-compliance compile 2>&1 > actual-errors.json

# Assert exit code is 1
echo $? # Expected: 1

# Assert error JSON matches
diff -u test/expected/04-invalid-missing-owner.errors.json actual-errors.json
```

7.6 Automating Tests with a Makefile

Create a Makefile target to run all tests:

```
.PHONY: test
test: test-lowering test-validation

.PHONY: test-lowering
test-lowering:
    @echo "Testing lowering..."
    @for fixture in test/fixtures/*.yaml; do \
        name=$(basename $$fixture .yaml); \
        if [ -f test/expected/$$name.lowered.yaml ]; then \
            cat $$fixture | ./bin/gert-kit-compliance compile > /tmp/$$name.actual.yaml; \
            diff -u test/expected/$$name.lowered.yaml /tmp/$$name.actual.yaml || exit 1; \
            echo "✓ $$name"; \
        fi \
    done

.PHONY: test-validation
test-validation:
    @echo "Testing validation..."
    @for fixture in test/fixtures/04-invalid-*.yaml; do \
        name=$(basename $$fixture .yaml); \
        cat $$fixture | ./bin/gert-kit-compliance compile > /tmp/$$name.actual.json 2>&1; \
        if [ $$? -eq 1 ]; then \
            diff -u test/expected/$$name.errors.json /tmp/$$name.actual.json || exit 1; \
            echo "✓ $$name"; \
        else \
            echo "× $$name: expected exit code 1, got 0"; \
            exit 1; \
        fi \
    done
```

Run with:

```
make test
```

7.7 Continuous Integration

Run Kit tests in CI (GitHub Actions, GitLab CI, etc.) to ensure regressions are caught before merge.

Example GitHub Actions workflow:

```
name: Kit Tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Build Kit compiler
        run: go build -o bin/gert-kit-compliance ./cmd/compiler
      - name: Run tests
        run: make test
```

7.8 Versioning Test Fixtures

As your Kit evolves, maintain backward compatibility by:

1. Keeping test fixtures for all major versions (e.g., `test/v1/fixtures/`, `test/v2/fixtures/`).
2. Ensuring the compiler produces correct output for older runbooks (if you support multiple Kit versions).
3. Adding new fixtures for new features without breaking old fixtures.

This ensures that users can upgrade the Kit without rewriting their runbooks.

Chapter 8

Publishing

8.1 Local-First Model

Gert Domain Kits follow a **local-first** distribution model. There is no central Kit registry, no cloud service, and no telemetry. A Kit is a directory containing a manifest and binaries. It can be:

- Checked into a Git repository alongside runbooks.
- Distributed as a tarball (`.tar.gz`).
- Installed from a package manager (`apt`, `brew`, `npm`, etc., if packaged).
- Symlinked from a shared filesystem.
- Vendored into a project directory.

This aligns with gert's local-first philosophy: runbooks and their dependencies should be portable, versionable, and executable without network access.

8.2 Referencing a Kit from a Runbook

Runbooks reference a Kit in two places:

1. In the `apiVersion` field:

`apiVersion: runbook/v2+compliance/v1`

The suffix (`compliance/v1`) tells gert which Kit schema to load. This is declared in the Kit manifest under `schema.apiVersion-suffix`.

2. In the `meta.kit` field (optional but recommended):

```
meta:
  title: "Quarterly access review"
kit:
  name: gert.compliance      # Kit identifier
  version: "^1.2.0"          # Semver range
```

The `meta.kit` field allows explicit version pinning. If the installed Kit version does not satisfy the constraint, `gert validate` rejects the runbook.

8.3 Kit Discovery

When `gert` encounters a runbook with `apiVersion: runbook/v2+<suffix>`, it searches for the corresponding Kit in the following order:

1. **Project-local Kits:** `./kits/<kit-name>/`
2. **User Kits:** `~/.gert/kits/<kit-name>/`
3. **GERT_KIT_PATH directories:** Each directory in the colon-separated `GERT_KIT_PATH` environment variable (similar to `PATH`).
4. **System Kits:** `/usr/local/share/gert/kits/<kit-name>/` (platform-specific).

Resolution algorithm:

1. Extract the Kit identifier from `apiVersion-suffix`. Example: `compliance/v1` → look for a Kit whose `schema.apiVersion-suffix` is `compliance/v1`.
2. Search each directory for a `gert-kit.yaml` manifest.
3. Load the manifest and check:
 - Does `schema.apiVersion-suffix` match?
 - If `meta.kit.version` is specified in the runbook, does the Kit version satisfy the constraint?
 - Is the Kit compatible with this `gert` core version (`compatibility.gert-core`)?
4. If a match is found, load the Kit. Otherwise, emit an error: “Kit not found: compliance/v1”.

8.4 Packaging

A Kit is a directory. The simplest packaging is a tarball:

```
tar -czf gert.compliance-1.2.0.tar.gz gert.compliance/
```

Users extract it:

```
tar -xzf gert.compliance-1.2.0.tar.gz -C ~/.gert/kits/
```

Gert will discover the Kit at `~/.gert/kits/gert.compliance/`.

Platform-specific binaries: If your Kit compiler is written in Go, compile binaries for multiple platforms:

```
GOOS=linux GOARCH=amd64 go build -o bin/gert-kit-compliance-linux-amd64 ./cmd/compiler
GOOS=darwin GOARCH=arm64 go build -o bin/gert-kit-compliance-darwin-arm64 ./cmd/compiler
GOOS=windows GOARCH=amd64 go build -o bin/gert-kit-compliance-windows-amd64.exe ./cmd/compiler
```

Ship all binaries in the Kit directory. Gert selects the appropriate binary at runtime based on `GOOS/GOARCH`.

Alternative: Universal binaries with gert kit install: If gert provides a `gert kit install` command in the future, it could:

```
gert kit install gert.compliance@1.2.0 --from https://github.com/acme/gert-kit-compliance/releases
```

This is future work. For now, users install Kits manually.

8.5 Versioning and Pinning

Semantic versioning: Kit versions MUST follow semver:

1.0.0 — Initial stable release.

1.1.0 — Added new step type (backward-compatible).

1.2.0 — Added optional field to existing step type (backward-compatible).

2.0.0 — Renamed required field (breaking change).

Pinning in runbooks: Exact pin:

```
meta:
  kit:
    name: gert.compliance
    version: "1.2.0"           # Must be exactly 1.2.0
```

Range pin (recommended):

```
meta:
  kit:
    name: gert.compliance
    version: "^1.2.0"         # Any 1.x >= 1.2.0
```

The caret (^) allows minor and patch updates but blocks major updates.

Validation: `gert validate` checks that the installed Kit version satisfies the pin:

```
$ gert validate runbook.yaml
Error: Kit version mismatch
  Required: gert.compliance ^1.2.0
  Installed: gert.compliance 2.0.0
  Location: ~/.gert/kits/gert.compliance/
```

8.6 Distribution Options

Option 1: Git submodule Vendor the Kit into your runbook repository:

```
git submodule add https://github.com/acme/gert-kit-compliance kits/gert.compliance
git submodule update --init
```

Gert will discover the Kit at `./kits/gert.compliance/`.

Option 2: Local path Copy the Kit directory into your project:

```
cp -r ~/Downloads/gert.compliance-1.2.0 ./kits/gert.compliance
```

Option 3: Archived bundle Distribute a tarball via GitHub Releases, a web server, or internal artifact repository:

```
curl -L https://github.com/acme/gert-kit-compliance/releases/download/v1.2.0/gert.compliance-  
tar -xz -C ~/.gert/kits/
```

Option 4: Package manager For broader distribution, package the Kit for `apt`, `brew`, `npm`, etc.

Example `brew` formula:

```
class GertKitCompliance < Formula  
  desc "Compliance and audit-focused Domain Kit for gert"  
  homepage "https://github.com/acme/gert-kit-compliance"  
  url "https://github.com/acme/gert-kit-compliance/archive/v1.2.0.tar.gz"  
  sha256 "..."  
  
  def install  
    bin.install "bin/gert-kit-compliance"  
    prefix.install "gert-kit.yaml", "schemas"  
  end  
end
```

Users install with:

```
brew install gert-kit-compliance
```

The formula installs binaries to `/usr/local/bin/` and Kit artifacts to `/usr/local/share/gert/kits/gert.`

8.7 Updating a Kit

For runbook authors:

1. Check the Kit changelog to see what changed.
2. Update the `meta.kit.version` pin in your runbook.
3. Run `gert validate` to ensure the runbook is still valid.
4. If the Kit has breaking changes (major version bump), update your runbook syntax to match the new schema.

For Kit developers:

1. Bump the version in `gert-kit.yaml`.
2. Update the changelog.
3. Tag the release in Git: `git tag v1.3.0 && git push -tags`.
4. Build and package binaries for all platforms.
5. Publish the tarball to GitHub Releases or your distribution mechanism.

8.8 Example: Publishing Checklist

1. **Test:** Run `make test` to ensure all fixtures pass.
2. **Version bump:** Update `meta.version` in `gert-kit.yaml`.
3. **Changelog:** Document changes in `CHANGELOG.md`.
4. **Build binaries:** Compile for Linux, macOS, Windows.
5. **Tag release:** `git tag v1.2.0 && git push -tags`.
6. **Package:** `tar -czf gert.compliance-1.2.0.tar.gz gert.compliance/`
7. **Publish:** Upload tarball to GitHub Releases.
8. **Announce:** Notify users via documentation, blog post, or release notes.

Chapter 9

Reference

9.1 Kit Manifest Field Reference

This section provides a complete reference for all fields in the `gert-kit.yaml` manifest.

Field	Required	Description
<code>apiVersion</code>	Yes	Must be <code>kit/v1</code> .
<code>meta.name</code>	Yes	Reverse-DNS namespaced identifier (e.g., <code>gert.compliance</code> , <code>com.acme.workflow</code>).
<code>meta.version</code>	Yes	Semantic version string (MUST follow semver, e.g., <code>"1.2.0"</code>).
<code>meta.description</code>	Yes	Human-readable summary of the Kit's purpose.
<code>meta.author</code>	No	Author name and contact (e.g., <code>"Acme Corp <kit@acme.example>"</code>).
<code>meta.homepage</code>	No	URL to Kit documentation or repository.
<code>meta.license</code>	No	SPDX license identifier (e.g., <code>"Apache-2.0"</code>).
<code>compatibility.gert-core</code>	No	Semver range specifying compatible gert core versions (e.g., <code>">=2.0.0 <3.0.0"</code>). If omitted, Kit claims compatibility with all versions.
<code>schema.apiVersion-suffix</code>	No	The suffix appended to <code>runbook/v2+</code> in runbooks authored with this Kit (e.g., <code>"compliance/v1"</code>).

Continued on next page

Field	Required	Description
<code>schema.json-schema</code>	No	Path (relative to Kit root) to the Kit's JSON Schema overlay (e.g., <code>./schemas/compliance-v1.json</code>).
<code>compiler.executable</code>	Yes	Path (relative to Kit root) to the compiler binary (e.g., <code>./bin/gert-kit-compliance</code>).
<code>compiler.args</code>	No	Array of arguments passed to the compiler (e.g., <code>["compile"]</code>). Default: no args.
<code>compiler.input-format</code>	No	Input format. Must be <code>yaml</code> . Default: <code>yaml</code> .
<code>compiler.output-format</code>	No	Output format. Must be <code>yaml</code> . Default: <code>yaml</code> .
<code>step-types</code>	No	Array of Kit-defined step types. Each entry has <code>name</code> , <code>lowers-to</code> , and <code>description</code> .
<code>step-types[].name</code>	Yes	Step type identifier (e.g., <code>approval-request</code>).
<code>step-types[].lowers-to</code>	Yes	Array of core step types this Kit type compiles into (e.g., <code>[manual, tool]</code>).
<code>step-types[].description</code>	No	Human-readable description of the step type.
<code>validators</code>	No	Array of domain-specific validators.
<code>validators[].name</code>	Yes	Validator identifier (e.g., <code>compliance/owner-required</code>).
<code>validators[].phase</code>	Yes	Validation phase: <code>parse</code> or <code>plan</code> .
<code>validators[].description</code>	No	Human-readable description of what the validator checks.
<code>projections</code>	No	Array of projections (read models over trace data).
<code>projections[].name</code>	Yes	Projection identifier (e.g., <code>compliance/audit-log</code>).
<code>projections[].description</code>	No	Human-readable description of the projection output.
<code>projections[].output-format</code>	No	Output format: <code>json</code> , <code>yaml</code> , <code>markdown</code> , or <code>text</code> . Default: <code>json</code> .

9.2 Kit Compiler Binary Interface

9.2.1 Invocation

```
<compiler-binary> [args...] < input.yaml > output.yaml  
echo $? # Exit code
```

9.2.2 Arguments

Arguments are specified in the manifest under `compiler.args`. Common convention:

`compile` — Perform lowering (default command).

`validate` — Run validation only (optional, if validators are bundled into the compiler).

`-version` — Print Kit version and exit.

9.2.3 Input

Kit-authored runbook YAML on stdin. Example:

```
apiVersion: runbook/v2+compliance/v1  
meta:  
  title: "Example runbook"  
steps:  
  - id: step-1  
    type: approval-request  
    approvers: [alice@acme.example]  
    timeout: 1h
```

9.2.4 Output

On success (exit code 0): Lowered runbook/v2 YAML on stdout:

```
apiVersion: runbook/v2  
meta:  
  title: "Example runbook"  
x-kit:lowered-from:  
  kit: gert.compliance
```

```
    version: "1.2.0"
  steps:
  - id: step-1
    type: manual
    approval:
      required: true
      approvers: [alice@acme.example]
      timeout: 3600
```

On failure (exit code 1): JSON array of error messages on stdout:

```
[
  {
    "file": "<stdin>",
    "line": 5,
    "severity": "error",
    "message": "approval-request requires at least one approver",
    "rule": "compliance/lowering"
  }
]
```

9.2.5 Exit Codes

0 — Success. Stdout contains valid runbook/v2 YAML.

1 — Validation or lowering error. Stdout contains error JSON.

2+ — Internal compiler error. Stderr may contain stack trace.

9.3 Validator Binary Interface

9.3.1 Invocation

```
<validator-binary> < runbook.yaml > errors.json
echo $? # Exit code: 0 = pass, 1 = errors
```

9.3.2 Input

Parse-time validators: Kit-authored runbook YAML on stdin.

Plan-time validators: Lowered runbook/v2 YAML on stdin.

9.3.3 Output

JSON array of validation messages:

```
[
  {
    "file": "runbook.yaml",
    "line": 3,
    "column": 1,
    "severity": "error",
    "message": "Compliance runbooks must declare an owner in meta.roles.owner",
    "rule": "compliance/owner-required"
  },
  {
    "file": "runbook.yaml",
    "line": 15,
    "severity": "warning",
    "message": "Timeout exceeds recommended SLA",
    "rule": "compliance/timeout-sla"
  }
]
```

If validation passes, output an empty array:

```
[]
```

9.3.4 Exit Codes

0 — No errors (warnings may be present).

1 — At least one error.

2+ — Internal validator error.

1+ — Projection error (malformed input, internal error).

9.5 Environment Variables

`GERT_KIT_PATH` — Colon-separated list of directories to search for Kits (similar to `PATH`).

Example: `/usr/local/share/gert/kits:~/.gert/kits`.

`GERT_KIT_DEBUG` — If set to 1, gert emits debug logs about Kit discovery and loading.

`GERT_NO_KIT_CACHE` — If set to 1, gert does not cache lowered runbooks. Useful during Kit development.

9.6 Error Codes and Meanings

9.6.1 Compiler Exit Codes

0 — Lowering succeeded.

1 — Validation or lowering error (recoverable; error JSON on stdout).

2 — Internal compiler error (panic, unexpected failure).

9.6.2 Validator Exit Codes

0 — Validation passed (no errors).

1 — Validation failed (at least one error).

2 — Internal validator error.

9.6.3 Projection Exit Codes

0 — Projection succeeded.

1 — Malformed input (invalid NDJSON).

2 — Internal projection error.

9.7 Trace Event Schema Reference

All trace events have the following common fields:

`event` (string) — Event type (e.g., `run.started`, `step.completed`).

`timestamp` (string) — ISO 8601 timestamp (UTC).

`run_id` (string) — Unique run identifier.

`step_id` (string, optional) — Step ID (for step-level events).

`payload` (object) — Event-specific data.

9.7.1 Event Types

Event Type	Payload Fields
<code>run.started</code>	<code>runbook_path</code> : string (path to runbook file)
<code>run.completed</code>	<code>status</code> : <code>success</code> <code>failed</code> <code>cancelled</code>
<code>step.started</code>	(empty)
<code>step.completed</code>	<code>exit_code</code> : integer, <code>duration_ms</code> : integer
<code>step.failed</code>	<code>error</code> : string (error message)
<code>evidence.captured</code>	<code>type</code> : <code>text</code> <code>attachment</code> , <code>value</code> : string, <code>sha256</code> : string (for attachments)
<code>approval.requested</code>	<code>approvers</code> : array of strings
<code>approval.granted</code>	<code>approver</code> : string (who approved)
<code>approval.rejected</code>	<code>approver</code> : string, <code>reason</code> : string
<code>variable.set</code>	<code>name</code> : string, <code>value</code> : any

9.8 Common Pitfalls

Pitfall 1: Non-deterministic lowering **Problem:** Compiler uses timestamps, random IDs, or environment variables in lowered output.

Solution: Make the compiler pure. If you need unique IDs, derive them deterministically from input data (e.g., hash of step ID + runbook title).

Pitfall 2: Validators with side effects **Problem:** Validator calls an API to check if a user exists.

Solution: Validators MUST be pure. If you need runtime checks, use a **Tool** or **Extension**, not a validator.

Pitfall 3: Forgetting to handle validation phase **Problem:** Parse-time validator tries to validate cross-step invariants (which requires seeing the full lowered runbook).

Solution: Use `phase: plan` for cross-step validators. They run after lowering.

Pitfall 4: Breaking changes without major version bump **Problem:** Kit removes a field in version 1.3.0 (should be 2.0.0).

Solution: Follow semver strictly. Removing a field is a breaking change → major version bump.

Pitfall 5: Not testing lowered output against core schema **Problem:** Compiler produces invalid `runbook/v2` YAML that gert rejects.

Solution: Always validate lowered output against the core `runbook/v2` JSON Schema as part of your test suite.